Interthread communication

```
class Account {

	private int availableBalance = 0;

	synchronized void withdraw(int amount) {
		// 1.account object is locked here
		System.out.println("Employee is going to withdraw the
money: " + amount);

		if (availableBalance < amount)// 0<500 true
		{
			System.out.println("Not enough balance. Waiting for
deposit...");
			try {
				// 2.waiting and releases the account object
lock
				wait();
				// 5 employee thread .receives the
notification signal. eligible to run.
				// 5.now employee thread goes from waiting to
blocked because this method is
				// synchronized.

			} catch (InterruptedException e) {
				e.printStackTrace();
			}
		}
		// Employee thread re-acquires the account lock

		// after getting notification, starts executing after
wait method

		if (availableBalance > amount) {
			availableBalance = availableBalance - amount;
			System.out.println("Withdrawal successful! Remaining
balance: " + availableBalance);
		} else {
			System.out.println("Not enough balance");
		}

	}
	//

   //account object l
	synchronized void deposit(int amount) {
		// finance thread acquires the lock
		// 3.finance thread acquires the lock because already
wait released the lock
```

```
            System.out.println("Depositing: " + amount);

            availableBalance = availableBalance + amount;

            //just sends the notification- just waking up the
employee thread.
            //not releasing the account object lock
            notify();
            // 4.sends the wakeup signal to waiting thread.still
finance thread holds the
            // lock

      }
      // finance thread releases the lock on the account object
}

class Employee extends Thread {

      private Account account;

      public Employee(Account account) {
            this.account = account;
      }

      public void run() {
            account.withdraw(500);
      }
}

class Finance extends Thread {

      private Account account;

      public Finance(Account account) {
            this.account = account;
      }

      public void run() {
            account.deposit(1000);
      }
}

public class EmployeeDemo {

      public static void main(String[] args) {
            Account account = new Account();

            Employee employee = new Employee(account);//employee
thread
```

```
            Finance finance = new Finance(account);//finance person

            employee.start();
    //80
            try {
                  Thread.sleep(1000);
            }

            catch (InterruptedException e) {
                  e.printStackTrace();

            }

            finance.start();
      }
}
```


ACCOUNT OBJECT
availableBalance = 0
Employee Thread
Finance Thread
account.withdraw(500)
Acquires Account lock
availableBalance < 500
wait()
RELEASES Account lock
WAITING
account.deposit(1000)
Acquires Account lock
availableBalance = 1000
notify()
----- Employee is WOKEN UP ------
BLOCKED
(waiting to get Account lock)
deposit() continues
deposit() finishes
Acquires Account lock
RELEASES Account lock
wait() returns
Continues AFTER wait()
availableBalance = 1000
Withdraws 500
Remaining balance = 500
Releases Account lock

Without Thread pool

```
class FoodOrder implements Runnable {

    private int orderNumber;

    FoodOrder(int orderNumber) {
        this.orderNumber = orderNumber;
    }

    @Override
    public void run() {

        System.out.println(
            Thread.currentThread().getName()
            + " is processing Order " + orderNumber
        );

        //menu preparation time
        try {
            Thread.sleep(2000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println(
            Thread.currentThread().getName()
            + " completed Order " + orderNumber
        );
    }
}

public class WithoutThreadPool {

    public static void main(String[] args) {

        // Create 6 Order objects - 6 tasks
      FoodOrder order1 = new FoodOrder(1);
      FoodOrder order2 = new FoodOrder(2);
      FoodOrder order3 = new FoodOrder(3);
      FoodOrder order4 = new FoodOrder(4);
      FoodOrder order5 = new FoodOrder(5);
      FoodOrder order6 = new FoodOrder(6);

        // Create 6 Thread objects
        Thread t1 = new Thread(order1);
        Thread t2 = new Thread(order2);
        Thread t3 = new Thread(order3);
        Thread t4 = new Thread(order4);
        Thread t5 = new Thread(order5);
```

```
        Thread t6 = new Thread(order6);

        // Start the threads
        t1.start();
        t2.start();
        t3.start();
        t4.start();
        t5.start();
        t6.start();
    }
}
```

With thread pool

```
import java.util.concurrent.ExecutorService;

import java.util.concurrent.Executors;


class Order implements Runnable {


   private int orderNumber;


   Order(int orderNumber) {

      this.orderNumber = orderNumber;

   }


   @Override

   public void run() {


      System.out.println(

         Thread.currentThread().getName()

         + " is processing Order " + orderNumber

      );


      try {

         Thread.sleep(2000);
```

```
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println(
            Thread.currentThread().getName()
            + " completed Order " + orderNumber
        );
    }
}

public class FoodDeliveryMain {
    public static void main(String[] args) {
        ExecutorService pool =
                    //method that creates/manages a fixed-size thread pool
                    //Create a thread pool that is configured to use a maximum of 3 worker threads.
                Executors.newFixedThreadPool(3);



        // Create and submit 6 orders
        pool.submit(new Order(1));//task1
        pool.submit(new Order(2));//task2
        pool.submit(new Order(3));//task3
        pool.submit(new Order(4));//task4
        pool.submit(new Order(5));//task5
        pool.submit(new Order(6));//task6

        // Stop accepting new tasks
        pool.shutdown();
    }
```

```
}
```

Thread priority

```
class PriorityWorker extends Thread {
    public PriorityWorker(String name) {
        super(name);
    }

    @Override
    public void run() {
        System.out.println(getName() + " [Priority: " +
getPriority() + "] is running.");
    }
}

public class ThreadPriorityDemo {
    public static void main(String[] args) {
        PriorityWorker lowPriorityThread = new PriorityWorker("Low-
Priority-Thread");
        PriorityWorker normalPriorityThread = new
PriorityWorker("Normal-Priority-Thread");
        PriorityWorker highPriorityThread = new
PriorityWorker("High-Priority-Thread");

        // Set explicit thread priorities
        lowPriorityThread.setPriority(Thread.MIN_PRIORITY);   //
Priority: 1
        normalPriorityThread.setPriority(Thread.NORM_PRIORITY); //
Priority: 5 (Default)
        highPriorityThread.setPriority(Thread.MAX_PRIORITY);  //
Priority: 10

        // Start all threads
        lowPriorityThread.start();
        normalPriorityThread.start();
        highPriorityThread.start();
    }
}
```